CUDA Thread Synchronization - 동기화의 모든 것

학습 목표

- `__syncthreads()` 완벽 이해
- `__syncwarp()` 사용법
- Atomic operations 마스터
- Memory fence 개념
- Race condition 회피

예상 학습 시간: 3-4시간
난이도: ⭐ ⭐ (중급)
선수 과목: 00-02 CUDA 기초

---

1. 왜 동기화가 필요한가?

1.1 Race Condition 문제

```
// 문제 상황: 2개 threads가 같은 변수 수정
__global__ void race_condition_bug() {
    __shared__ int counter;

    if (threadIdx.x == 0) {
        counter = 0;  // Thread 0이 초기화
    }

    // ❌ BUG: Thread 1이 counter를 읽을 때 초기화 안 됐을 수 있음!
    counter++;  // Race condition!
}

// 실행 순서가 보장되지 않음:
// 가능한 시나리오 1:
```

```
//   Thread 0: counter = 0
//   Thread 1: counter++ (정상 동작)

// 가능한 시나리오 2:
//   Thread 1: counter++ (garbage 값!)
//   Thread 0: counter = 0
```

1.2 Shared Memory 협업 패턴

```
// 전형적인 패턴: Load → Process → Store
__global__ void typical_pattern(float* input, float* output) {
    __shared__ float tile[256];
    int tid = threadIdx.x;

    // Phase 1: Load data into shared memory
    tile[tid] = input[blockIdx.x * 256 + tid];

    // ⚠️ 동기화 필요! 모든 threads가 load 완료 대기
    __syncthreads();

    // Phase 2: Process (다른 thread 데이터 사용)
    float result = tile[tid] + tile[(tid + 1) % 256];

    // ⚠️ 다시 동기화! 모든 threads가 process 완료 대기
    __syncthreads();

    // Phase 3: Store result
    output[blockIdx.x * 256 + tid] = result;
}
```

---

2. Block-Level Synchronization: `__syncthreads()`

2.1 기본 사용법

```
__syncthreads();  // Block 내 모든 threads가 여기서 대기
```

동작 방식:

1. 각 thread가 `__syncthreads()` 에 도달

2. Block의 **모든 threads가 도달할 때까지 대기**
3. 모든 threads 동시에 다음 명령어로 진행

제약 조건:

- ❌ Conditional 안에서 사용 불가 (warp divergence)
- ✅ Block 내에서만 동작 (다른 blocks와는 동기화 안 됨)

2.2 올바른 사용 예시

```
__global__ void correct_usage(float* data) {
    __shared__ float shared[256];
    int tid = threadIdx.x;

    // ✅ Good: 모든 threads가 실행
    shared[tid] = data[tid];
    __syncthreads();

    float val = shared[(tid + 1) % 256];
    __syncthreads();

    data[tid] = val;
}
```

2.3 잘못된 사용 예시

```
__global__ void wrong_usage(float* data) {
    __shared__ float shared[256];
    int tid = threadIdx.x;

    shared[tid] = data[tid];

    // ❌ WRONG: Conditional 안에서 사용
    if (tid < 128) {
        __syncthreads();  // Deadlock! tid >= 128인 threads는 도달 안 함
    }

    // ❌ WRONG: Divergent path
    if (tid % 2 == 0) {
        __syncthreads();  // 절반만 도달
    } else {
        __syncthreads();  // 나머지 절반만 도달
```

```
    }
    // → 두 __syncthreads()는 다른 barrier!
}

// 올바른 수정:
__global__ void correct_version(float* data) {
    __shared__ float shared[256];
    int tid = threadIdx.x;

    shared[tid] = data[tid];

    // ✅ Good: 조건 밖에서 동기화
    __syncthreads();

    if (tid < 128) {
        // 안전하게 shared memory 사용
        shared[tid] += shared[tid + 128];
    }

    __syncthreads();  // 다시 동기화
}
```

2.4 성능 고려사항

```
// __syncthreads() 비용: ~수십 cycles

// ❌ Bad: 불필요한 동기화
__syncthreads();
__syncthreads();  // 연속된 동기화는 비효율적
__syncthreads();

// ✅ Good: 필요할 때만
shared[tid] = data[tid];
__syncthreads();  // 여기만 필요!
result = process(shared);
```

---

3. Warp-Level Synchronization

3.1 `__syncwarp()` - Warp 내부 동기화

```
__syncwarp();  // 현재 warp의 32 threads만 동기화
__syncwarp(0xffffffff);  // mask 지정 (모든 lanes)
```

언제 사용?

- Warp 내에서만 데이터 공유할 때
- `__syncthreads()` 보다 **훨씬 빠름** (~수 cycles)

```
__global__ void warp_sync_example(float* data) {
    int tid = threadIdx.x;
    int lane = tid % 32;  // Warp 내 lane ID

    float val = data[tid];

    // Warp 내에서만 shuffle (동기화 필요 없음)
    float neighbor = __shfl_xor_sync(0xffffffff, val, 1);

    // 하지만 명시적 동기화도 가능
    __syncwarp();

    data[tid] = neighbor;
}
```

3.2 Warp Shuffle - 레지스터 간 직접 통신

```
// Shuffle functions (암묵적 동기화 포함)
__shfl_sync(mask, var, srcLane);       // 특정 lane에서 읽기
__shfl_up_sync(mask, var, delta);      // 위쪽 lane에서 읽기
__shfl_down_sync(mask, var, delta);    // 아래쪽 lane에서 읽기
__shfl_xor_sync(mask, var, laneMask);  // XOR lane에서 읽기
```

예시: Warp Reduction

```
__device__ float warp_reduce_sum(float val) {
    // Tree-based reduction within warp (no shared memory!)
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;  // lane 0에 최종 결과
}

// 시각화 (8 threads 예시):
```

```
// Initial:  [1, 2, 3, 4, 5, 6, 7, 8]
// offset=4: [6, 8, 10, 12, 5, 6, 7, 8]  (0+=4, 1+=5, ...)
// offset=2: [16, 20, 10, 12, 5, 6, 7, 8] (0+=2, 1+=3, ...)
// offset=1: [36, 20, 10, 12, 5, 6, 7, 8] (0+=1)
// Result: thread 0 = 36 (1+2+3+4+5+6+7+8)
```

---

4. Atomic Operations

4.1 기본 Atomic 함수들

```
// Global/shared memory에 대한 atomic 연산
atomicAdd(address, val);    // *address += val (원자적)
atomicSub(address, val);    // *address -= val
atomicMin(address, val);    // *address = min(*address, val)
atomicMax(address, val);    // *address = max(*address, val)
atomicExch(address, val);   // *address = val, 이전 값 반환
atomicCAS(address, compare, val);  // Compare-And-Swap
```

지원 타입:

- `int`, `unsigned int`, `unsigned long long`
- `float` (제한적 - add, exch만)
- `double` (Compute Capability 6.0+)

4.2 사용 예시: Histogram

```
__global__ void histogram_atomic(int* data, int* hist, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < N) {
        int bin = data[tid] % 256;  // 0-255 범위

        // ✅ Atomic으로 안전하게 증가
        atomicAdd(&hist[bin], 1);

        // ❌ 이렇게 하면 race condition:
        // hist[bin]++;  // 여러 threads가 동시 접근!
```

```
    }
}
```

4.3 성능 고려사항

```
// Atomic은 느림! (~100-200 cycles, serialized)

// ❌ Bad: Global atomic이 병목
__global__ void slow_sum(float* data, float* result, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {
        atomicAdd(result, data[tid]);  // 모든 threads가 경쟁!
    }
}

// ✅ Good: Shared memory로 줄이고, block당 1번만 global atomic
__global__ void fast_sum(float* data, float* result, int N) {
    __shared__ float shared[256];
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int ltid = threadIdx.x;

    // Load
    shared[ltid] = (tid < N) ? data[tid] : 0.0f;
    __syncthreads();

    // Reduce within block (no atomics)
    for (int s = 128; s > 0; s >>= 1) {
        if (ltid < s) {
            shared[ltid] += shared[ltid + s];
        }
        __syncthreads();
    }

    // Only thread 0 does atomic
    if (ltid == 0) {
        atomicAdd(result, shared[0]);  // Block당 1번만!
    }
}

// 성능 차이: 100x+ faster! 🚀
```

4.4 Double Precision Atomic (Custom)

```
// float는 atomicAdd 지원, 하지만 double은?
#if __CUDA_ARCH__ < 600
__device__ double atomicAdd_double(double* address, double val) {
    unsigned long long int* address_as_ull = (unsigned long long int*)address;
    unsigned long long int old = *address_as_ull, assumed;

    do {
        assumed = old;
        old = atomicCAS(address_as_ull, assumed,
                        __double_as_longlong(val + __longlong_as_double(assumed)
    } while (assumed != old);

    return __longlong_as_double(old);
}
#endif
```

---

5. Memory Fence

5.1 Memory Fence 종류

```
__threadfence_block();  // Block 내 memory 연산 순서 보장
__threadfence();         // Grid 전체 (모든 blocks)
__threadfence_system();  // Host memory까지 포함
```

언제 사용?

- Memory 연산 순서를 명시적으로 보장해야 할 때
- Lock-free 알고리즘 구현 시

5.2 사용 예시

```
__global__ void producer_consumer(int* flag, int* data) {
    if (threadIdx.x == 0) {
        // Producer
        data[0] = 42;
        __threadfence();  // data 쓰기가 먼저 완료됨을 보장
        flag[0] = 1;      // 이후 flag 설정
    } else {
```

```
        // Consumer
        while (flag[0] == 0);  // Spin wait
        __threadfence();       // flag 읽기 후 data 읽기 순서 보장
        int value = data[0];   // 안전하게 읽기
    }
}
```

---

6. 실습: Parallel Reduction (단계별 최적화)

6.1 Version 1: Naive (Interleaved)

```
__global__ void reduce_v1(float* input, float* output, int N) {
    __shared__ float shared[256];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;

    // Load
    shared[tid] = (i < N) ? input[i] : 0.0f;
    __syncthreads();

    // Reduction (interleaved addressing)
    for (int s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0) {  // ❌ Warp divergence!
            shared[tid] += shared[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        output[blockIdx.x] = shared[0];
    }
}

// 문제:
// - tid % (2*s)로 인한 심각한 divergence
// - 성능: ~5 GB/s
```

6.2 Version 2: Sequential Addressing

```
__global__ void reduce_v2(float* input, float* output, int N) {
    __shared__ float shared[256];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;

    shared[tid] = (i < N) ? input[i] : 0.0f;
    __syncthreads();

    // ✅ Sequential addressing (no divergence)
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            shared[tid] += shared[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        output[blockIdx.x] = shared[0];
    }
}

// 개선:
// - Divergence 제거
// - 성능: ~50 GB/s (10배 향상!)
```

6.3 Version 3: Warp Shuffle (최종)

```
__device__ float warp_reduce(float val) {
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}

__global__ void reduce_v3(float* input, float* output, int N) {
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;

    float val = (i < N) ? input[i] : 0.0f;

    // Warp-level reduction (no shared memory for this part!)
    val = warp_reduce(val);

    // Store warp results to shared memory
```

```
    __shared__ float warp_results[8];  // 256 threads = 8 warps
    int warp_id = tid / 32;
    int lane = tid % 32;

    if (lane == 0) {
        warp_results[warp_id] = val;
    }
    __syncthreads();

    // Final reduction by first warp
    if (tid < 8) {
        val = warp_results[tid];
        val = warp_reduce(val);
        if (tid == 0) {
            output[blockIdx.x] = val;
        }
    }
}

// 최종 성능: ~200 GB/s (40배 향상!) 🚀
```

---

7. 동기화 패턴 요약

7.1 선택 가이드

상황	사용할 동기화	비용
Block 내 shared memory 사용	`__syncthreads()`	~50 cycles
Warp 내에서만 데이터 공유	`__syncwarp()`	~5 cycles
Warp shuffle 사용	Implicit (함수 내장)	~2 cycles
Global counter 업데이트	`atomicAdd()`	~100-200 cycles
Memory 순서 보장	`__threadfence()`	~수십 cycles

7.2 Best Practices

```
// ✅ DO:
// 1. Warp primitives 최대 활용 (shuffle)
// 2. Shared memory로 atomic 경쟁 줄이기
// 3. 불필요한 __syncthreads() 제거
// 4. Sequential addressing 사용

// ❌ DON'T:
// 1. Conditional 안에 __syncthreads()
// 2. 과도한 atomic 사용
// 3. Divergent warp에서 동기화
```

---

8. 다음 단계

이제 실전 알고리즘을 구현할 준비가 되었습니다:

- 04_Reduction.md - Parallel reduction 완전 정복
- 05_Scan.md - Prefix sum (scan) 알고리즘

실습 예제:

- `examples/cuda/04_reduction_simple.cu` - Tree reduction
- `examples/cuda/04_reduction_optimized.cu` - Warp shuffle 활용

요약 체크리스트

- [ ] `__syncthreads()` 올바르게 사용 가능
- [ ] Warp divergence 회피 패턴 숙지
- [ ] `__syncwarp()` vs `__syncthreads()` 차이 이해
- [ ] Warp shuffle로 reduction 구현 가능
- [ ] Atomic operations 적재적소 사용
- [ ] Shared memory로 atomic 경쟁 줄이기 가능
- [ ] Memory fence 개념 이해

핵심 개념:

- `__syncthreads()` = Block barrier

- Warp shuffle = Fastest communication
- Atomic = Slow but safe
- Shared memory → Reduce atomics